

ANDROID PHONE MANAGEMENT SYSTEM

INTRODUCTION:

The Android Phone Management System is a console-based application developed using C++ to simulate the core functionalities of a mobile phone. This project focuses on implementing fundamental operations such as managing contacts, sending messages, and maintaining call history in an efficient manner.

The main objective of this system is to demonstrate the application of important data structures in real-life scenarios. Contacts are managed using a Linked List, which allows dynamic storage and easy insertion of records. Messaging functionality is implemented using a Queue, following the First-In-First-Out (FIFO) principle to ensure messages are processed in the order they are sent. Call history is maintained using a Stack, which follows the Last-In-First-Out (LIFO) principle, enabling quick access to the most recent calls.

The system provides a user-friendly, menu-driven interface that allows users to perform operations such as adding contacts, viewing contact details, sending messages, viewing and deleting messages, making calls, and checking call logs.

This project enhances the understanding of data structures and their practical applications while improving programming skills in C++ through structured and modular design.

ABSTRACT:

The Android Phone Management System is a C++ based console application designed to simulate basic mobile phone functionalities such as contact management, messaging, and call history tracking. The project demonstrates the practical implementation of fundamental data structures in a real-world scenario.

Contacts are stored using a Linked List to allow dynamic memory allocation and easy insertion. Messages are managed using a Queue following the First-In-First-Out (FIFO) principle, ensuring messages are processed in order. Call history is maintained using a Stack based on the Last-In-First-Out (LIFO) principle, providing quick access to recent calls.

The system offers a simple menu-driven interface that allows users to perform operations like adding contacts, sending messages, viewing message history, making calls, and checking call logs.

Overall, this project helps in understanding the importance and application of data structures while improving programming skills in C++.

ALGORITHM:

- ◆ Step 1: Start the program

- ◆ Step 2: Initialize variables

Set head = NULL (Contacts)

Set front = rear = -1 (Message Queue)

Set top = -1 (Call Stack)

- ◆ Step 3: Display Menu

Add Contact

View Contacts

Send Message

View Messages

Delete Message

Make Call

Call History

Delete Last Call

Exit

- ◆ Step 4: Read user choice

- ◆ Step 5: Perform operation using switch-case

--➤ Case 1: Add Contact

Input name and number

Create new node

Insert at beginning of linked list

--➤ Case 2: View Contacts

Traverse linked list

Display all contacts

--➤ Case 3: Send Message

Input message

Check if queue is full

Insert message into queue (rear++)

--➤ Case 4: View Messages

Check if queue is empty

Display messages from front to rear

--➤ Case 5: Delete Message

Check if queue is empty

Remove message from front (front++)

--► Case 6: Make Call

Input contact name

Push name into stack (top++)

--► Case 7: Call History

Check if stack is empty

Display calls from top to bottom

--► Case 8: Delete Last Call

Check if stack is empty

Pop last call (top--)

--► Case 9: Exit

Exit loop

--► Default

Display "Invalid Choice"

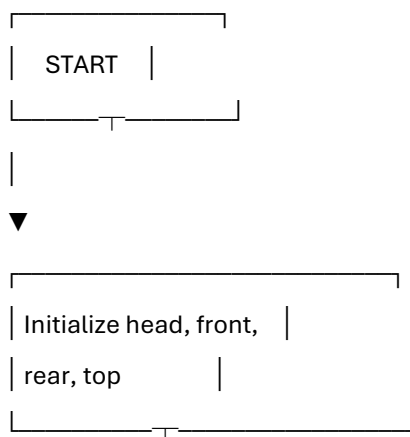
◆ Step 6: Repeat Steps 3–5 until user selects Exit

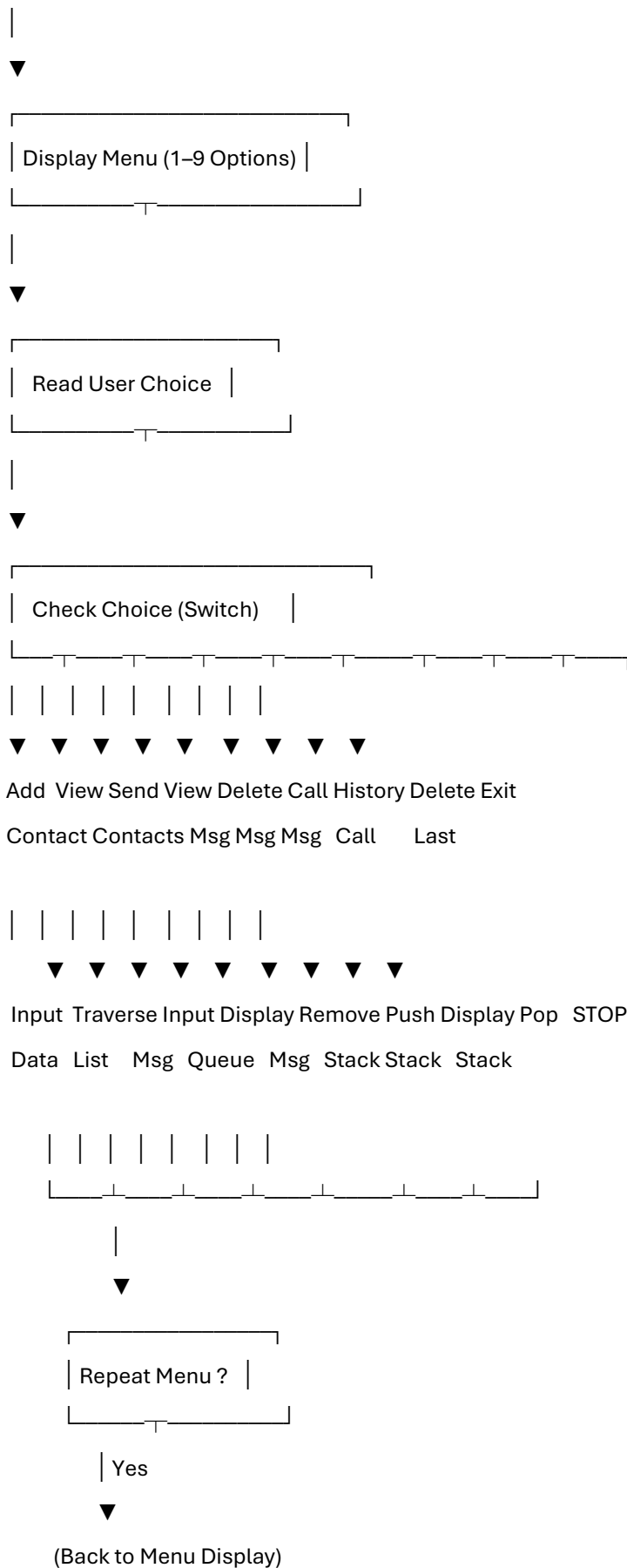
◆ Step 7: Free memory

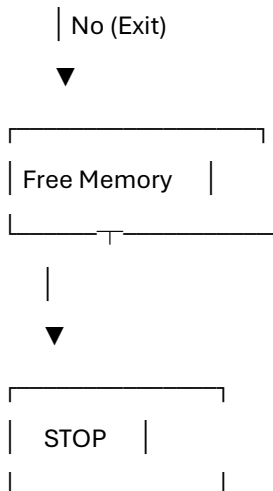
Delete all nodes in linked list

◆ Step 8: Stop the program

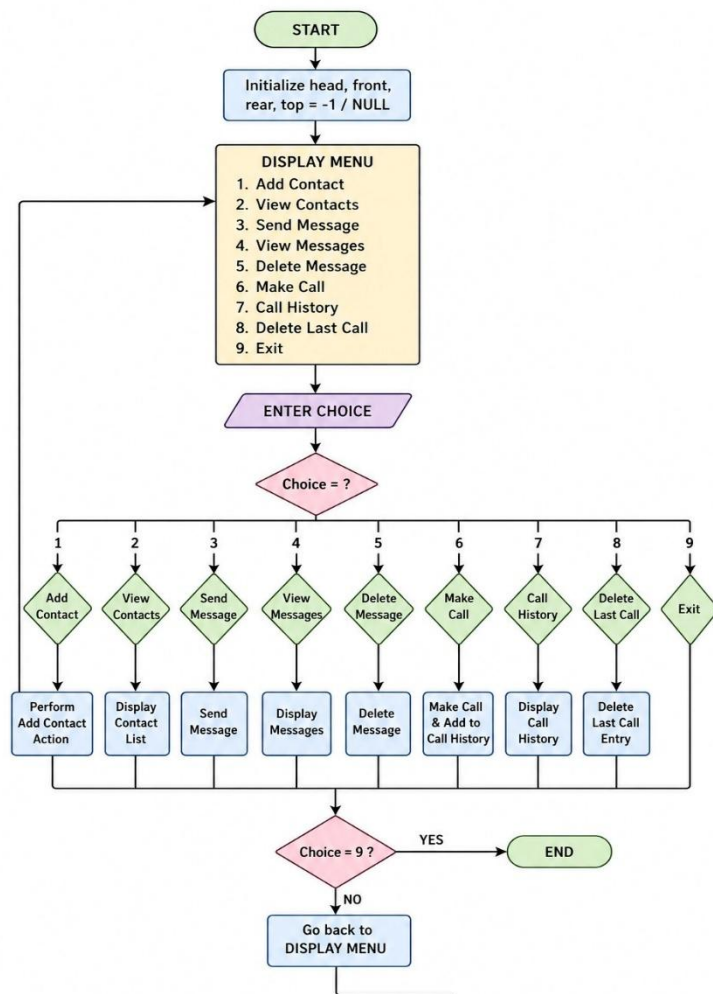
FLOWCHART :







- > ◆ Oval → Start/Stop
- > □ Rectangle → Process
- > ◇ Diamond → Decision
- > ➡ Arrows → Flow



METHODOLOGY:

◆ 1. System Design

- 1.The program is designed using a modular approach
- 2.Divided into three main modules:
- 3.Contacts (Linked List)
- 4.Messages (Queue)
- 5.Call History (Stack)

◆ 2. Data Structure Implementation

--> 📞 Contacts → Linked List

- 1.Dynamic memory allocation using new
- 2.Each contact stored as a node
- 3.Allows flexible addition of contacts

--> 💬 Messages → Queue (FIFO)

- 1.Implemented using array
- 2.Messages are stored in order
- 3.First message is deleted first

--> 🗃️ Call History → Stack (LIFO)

- 1.Implemented using array
- 2.Last call stored at top
- 3.Last call deleted first

◆ 3. User Interaction

- 1.Menu-driven system
- 2.User selects options (1–9)
- 3.Input taken using cin and getline()

◆ 4. Processing Logic

Uses:

- 1.Functions for each operation
- 2.Switch-case for menu control
- 3.Conditions (if-else) for validations

◆ 5. Memory Management

- 1.Contacts use dynamic memory
- 2.freeContacts() function deletes allocated memory
- 3.Prevents memory leakage

◆ 6. Program Flow

- 1.Start program
- 2.Display menu
- 3.Read user choice
- 4.Call respective function
- 5.Perform operation
- 6.Display result
- 7.Repeat until Exit
- 8.Free memory and stop

REQUIREMENTS:

✅ 1. Functional Requirements

(What the system should do)

--> 📞 Contacts Module (Linked List)

- 1.Add new contact (name & number)
- 2.Display all contacts
- 3.Store contacts dynamically

--> 💬 Messages Module (Queue)

- 1.Send/store messages
- 2.View all messages in order (FIFO)
- 3.Delete oldest message

--> 📞 Call History Module (Stack)

- 1.Store call history
- 2.Display recent calls (LIFO)
- 3.Delete last call

--> 🖥️ Main System

- 1.Menu-driven interface
- 2.Perform operations based on user choice
- 3.Exit option

✅ 2. Non-Functional Requirements

(How the system should behave)

- 1.Simple and easy-to-use interface
- 2.Fast execution
- 3.Efficient memory usage (dynamic allocation for contacts)
- 4.Organized code using functions and data structures

✅ 3. Hardware Requirements

- 1.Computer / Laptop
- 2.Minimum 2GB RAM
- 3.Keyboard for input

✅ 4. Software Requirements

- 1.C++ Compiler (GCC / Code::Blocks / Turbo C++)
- 2.Operating System (Windows / Linux / Mac)

✅ 5. Input Requirements

- 1.Contact name and number

- 2.Message text
- 3.Call name
- 4.Menu choice (1–9)

✅ 6. Output Requirements

- 1.Display contact list
- 2.Show messages
- 3.Show call history
- 4.Confirmation messages (Added / Deleted / Calling).

RESULT(SCREEN SHOTS):

```
Output
==== ANDROID PHONE SYSTEM ====
1. Add Contact
2. View Contacts
3. Send Message
4. View Messages
5. Delete Message
6. Make Call
7. Call History
8. Delete Last Call
9. Exit
Enter choice: 1
Enter Name: VIGNESH
Enter Number: 9100598298
Contact Added!
```

```
==== ANDROID PHONE SYSTEM ====
1. Add Contact
2. View Contacts
3. Send Message
4. View Messages
5. Delete Message
6. Make Call
7. Call History
8. Delete Last Call
9. Exit
Enter choice: 2
VIGNESH - 9100598298
```

```
==== ANDROID PHONE SYSTEM ====
1. Add Contact
2. View Contacts
3. Send Message
4. View Messages
5. Delete Message
6. Make Call
7. Call History
8. Delete Last Call
9. Exit
Enter choice: 3
Enter Recipient Name: VIGNESH
Enter Message: HELLO VIGNESH
Message Sent to VIGNESH!
```

```
==== ANDROID PHONE SYSTEM ====
1. Add Contact
2. View Contacts
3. Send Message
4. View Messages
5. Delete Message
6. Make Call
7. Call History
8. Delete Last Call
9. Exit
Enter choice: 4
1. To VIGNESH: HELLO VIGNESH
```



```
==== ANDROID PHONE SYSTEM ====
```

1. Add Contact
2. View Contacts
3. Send Message
4. View Messages
5. Delete Message
6. Make Call
7. Call History
8. Delete Last Call
9. Exit

Enter choice: 5

Deleting: To VIGNESH - HELLO VIGNESH

```
==== ANDROID PHONE SYSTEM ====
```

1. Add Contact
2. View Contacts
3. Send Message
4. View Messages
5. Delete Message
6. Make Call
7. Call History
8. Delete Last Call
9. Exit

Enter choice: 6

Enter Contact Name: VIGNESH

Calling VIGNESH...

```
==== ANDROID PHONE SYSTEM ====
```

1. Add Contact
2. View Contacts
3. Send Message
4. View Messages
5. Delete Message
6. Make Call
7. Call History
8. Delete Last Call
9. Exit

Enter choice: 7

VIGNESH

```
==== ANDROID PHONE SYSTEM ====
```

1. Add Contact
2. View Contacts
3. Send Message
4. View Messages
5. Delete Message
6. Make Call
7. Call History
8. Delete Last Call
9. Exit

Enter choice: 8

Deleting Call: VIGNESH

```
==== ANDROID PHONE SYSTEM ====
```

1. Add Contact
2. View Contacts
3. Send Message
4. View Messages
5. Delete Message
6. Make Call
7. Call History
8. Delete Last Call
9. Exit

Enter choice: 9

Exiting...

I

=== Code Execution Successful ===

CONCLUSION:

The Android Phone Management System successfully demonstrates the implementation of basic mobile phone functionalities using C++. This project highlights the effective use of fundamental data structures such as Linked List, Queue, and Stack to manage contacts, messages, and call history efficiently.

Through this system, operations like adding contacts, sending messages, and maintaining call logs are performed in a structured and organized manner. The use of appropriate data structures ensures better performance and logical handling of data based on real-world requirements.

Overall, the project provides a clear understanding of how data structures can be applied in practical applications. It also helps in improving programming skills, problem-solving ability, and knowledge of C++ concepts.

FUTURE ENHANCEMENTS (FUTURE WORK):

The current Android Phone Management System provides basic functionalities, but it can be further improved with additional features and enhancements. In future, the system can be upgraded by implementing a graphical user interface (GUI) to make it more user-friendly and interactive.

The contact management system can be enhanced by adding options such as editing and deleting contacts, searching with partial names, and storing additional details like email and address. Message functionality can be improved by implementing separate inbox and sent folders, along with timestamps for each message.

The system can also be extended to use file handling or databases to store data permanently, so that contacts, messages, and call history are not lost when the program is closed. Security features like password protection can be added to ensure user privacy.

Further improvements may include integrating advanced features such as notifications, call duration tracking, and sorting or filtering options for better data management.

Overall, these enhancements would make the system more realistic, efficient, and closer to an actual Android phone application.